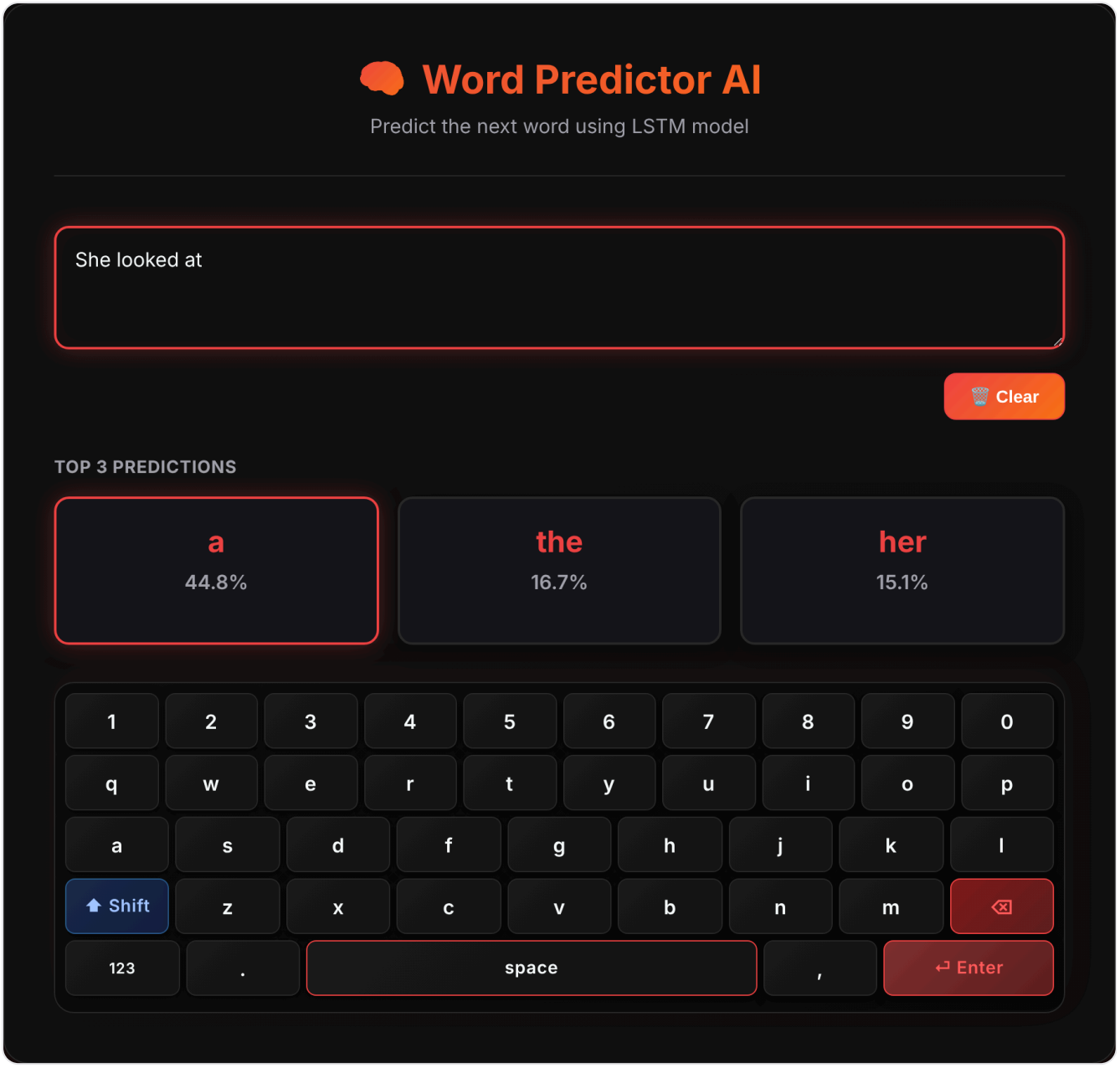
PROJECT REPORT · DEEP LEARNING / NLP

Word Predictor AI

A next-word prediction web app powered by an LSTM language model trained on classic literature, served through Flask with an on-screen keyboard interface.



LSTM
Model type

Keras 3 · TensorFlow
Framework

Flask + HTML/JS
Web stack

Version 1.1
October 2026

1 Overview

Word Predictor AI suggests the three most likely next words while the user types, similar to the suggestion bar on a phone keyboard. A recurrent neural network (LSTM) reads the last few words of the text and outputs a probability for every word in its vocabulary; the web interface shows the top three as clickable cards together with their probability.

5.19 M	10,000	9	Top 3
trainable parameters	words in active vocabulary	words of context	suggestions per request

Objectives

- Train a word-level language model on public-domain English literature.
- Expose the model through a small REST API (`POST /api/predict`).
- Build a responsive web UI with real-time suggestions and a virtual keyboard that works on desktop and mobile.

Features

- **Real-time predictions** — suggestions refresh 300 ms after the user stops typing.
- **Click to insert** — tapping a suggestion appends the word and immediately predicts the next one.
- **Virtual keyboard** — letters, numbers and symbols, with Shift (one capital) and double-tap Caps Lock.
- **Responsive** — the layout and keyboard adapt to phone screens.

2 Dataset and Preprocessing

The model is trained on `agentlans/project-gutenberg-top100` (English configuration) from the Hugging Face Hub — text from the 100 most downloaded Project Gutenberg books.

Step	What the notebook does
Sampling	Shuffle paragraphs (seed 42) and keep the first 4,000 non-empty ones.
Sentence split	Collapse whitespace, split on `. ! ?`, and keep sentences with at least 3 words (9,550 sentences).
Tokenization	Keras `Tokenizer(num_words=10000, oov_token='<OOV>')` : lower-cases text and strips punctuation. 19,687 distinct words are seen; only the 10,000 most frequent are kept and the rest become `<OOV>` .
n-gram sequences	Every sentence prefix becomes one training example: for “the sun was rising”, the inputs are “the → sun”, “the sun → was”, “the sun was → rising”.
Padding	Pre-pad / pre-truncate to length 10. The first 9 tokens are the input *x*; the last token is the label *y*.

3 Model Architecture

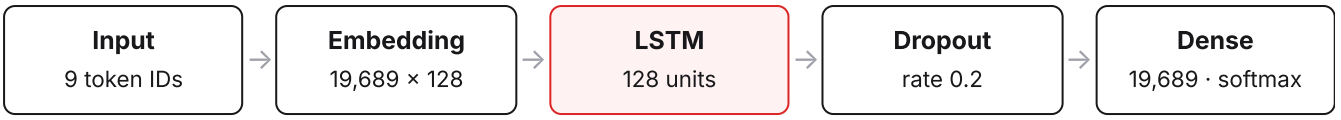


Figure 1. Sequential model. The LSTM's final hidden state summarises the context; the Dense layer turns it into a probability for every word.

Layer	Output shape	Parameters
Embedding (19,689 → 128)	`(None, 9, 128)`	2,520,192
LSTM (128 units)	`(None, 128)`	131,584
Dropout (0.2)	`(None, 128)`	0
Dense (softmax)	`(None, 19,689)`	2,539,881
Total		**5,191,657**

Training configuration

Setting	Value
Loss	Sparse categorical cross-entropy (integer labels, no one-hot encoding)
Optimizer	Adam, learning rate 0.001
Batch size / max epochs	128 / 100
Callbacks	`EarlyStopping(monitor='loss', patience=5, restore_best_weights=True)`, `ReduceLROnPlateau(factor=0.5, patience=3, min_lr=1e-5)`
Saved artefacts	`next_word_model.keras`, `tokenizer.pkl`, `model_config.pkl` (`max_sequence_len = 10`, `vocab_size = 19,689`)

4 System Design

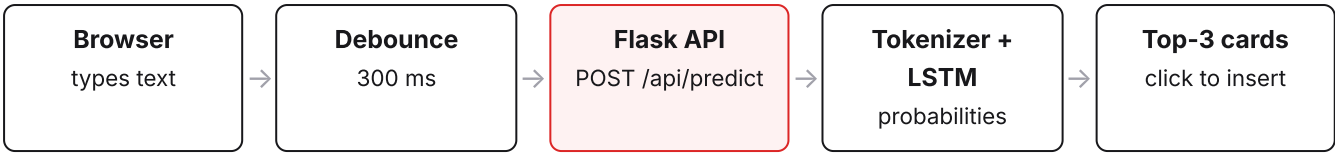


Figure 2. Request flow. The model and tokenizer are loaded once when the server starts.

Prediction steps (`app.py`)

1. Lower-case the text and convert words to IDs with the saved tokenizer (unknown words → `<OOV>`).
2. Keep the last 9 IDs and pre-pad with zeros — exactly as in training.
3. Run the model to get a probability for every vocabulary word.
4. Walk down the ranking, skipping padding, `<OOV>` and punctuation-only tokens, until 3 real words are found.
5. Return each word with its probability as a percentage.

API

Request

```
POST /api/predict
Content-Type: application/json

{ "text": "she looked at" }
```

Response

```
{ "predictions": [
  { "word": "a",   "confidence": 44.8 },
  { "word": "the", "confidence": 16.7 },
  { "word": "her", "confidence": 15.1 } ] }
```

Requests without a JSON object, or with a non-string `text` , get HTTP 400 with an error message. Empty text returns an empty list.

5 Results

Sample predictions

Input text	1st	2nd	3rd
“she looked at …”	a 44.8%	the 16.7%	her 15.1%
“they went to …”	the 57.5%	him 7.4%	us 3.9%
“he said that …”	he 35.9%	i 29.9%	we 4.6%
“i am …”	not 15.9%	going 8.4%	very 5.6%
“what is …”	it 22.9%	the 18.2%	your 9.3%
“my dear …”	child 12.9%	sir 10.9%	chettam 9.3%
“deep in the heart of the …”	dance 11.1%	castle 9.7%	garden 6.4%
“no i …”	never 14.6%	didn't 10.3%	don't 10.1%

The model reliably picks up grammar: after a preposition (“looked at”, “went to”) it proposes determiners and pronouns; after “I am” it proposes a negation, a verb or an adverb. It also picks up

phrases from the books it read, such as “my dear *child* / *sir*”, and even a character name (“Chettam”, from *Middlemarch*). Probabilities are naturally spread out because many words can follow a phrase, which is why the top suggestion often scores below 50%.

User interface

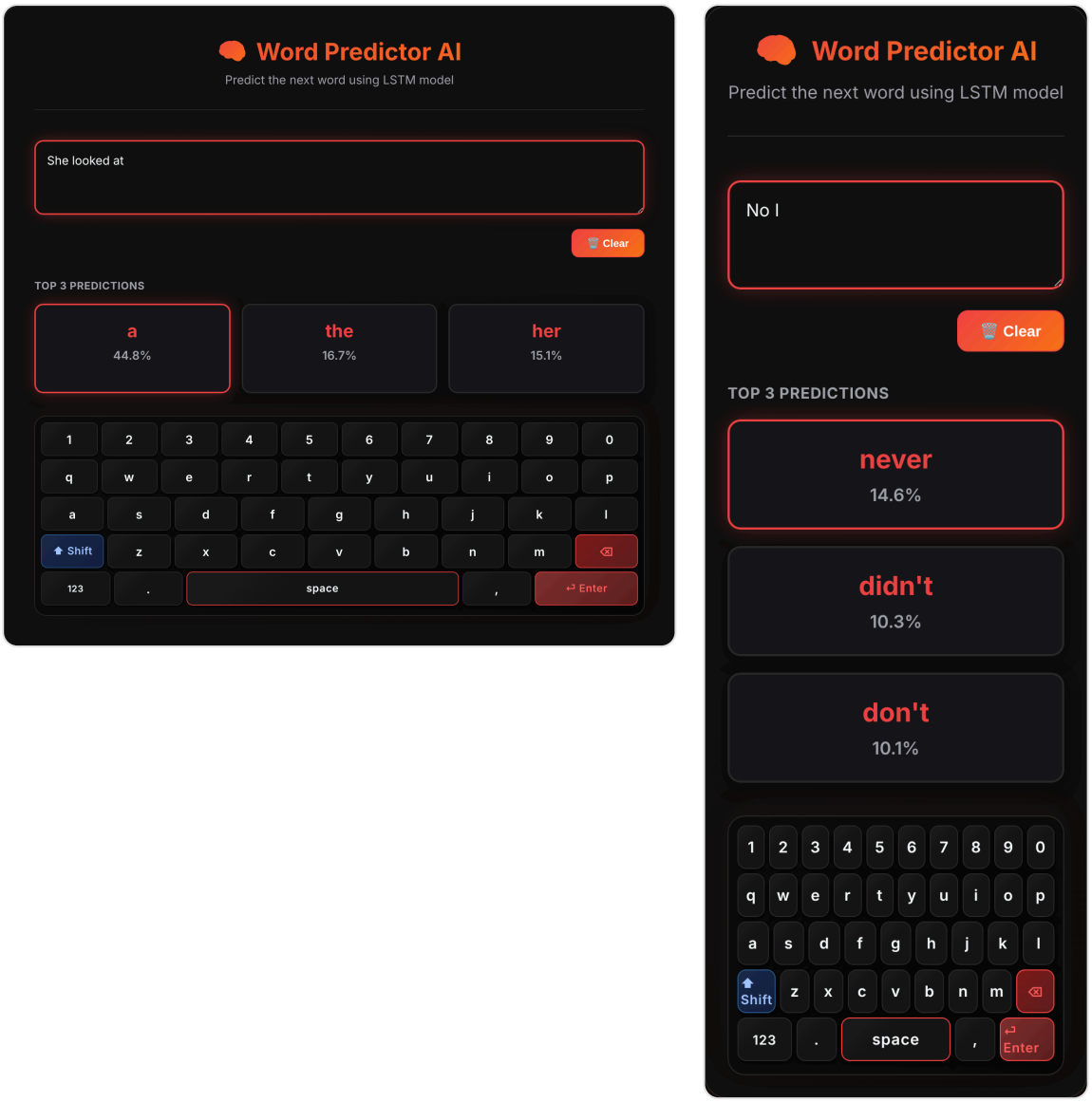


Figure 3. Desktop (left) and phone (right). On the phone, “No I” gives *never*, *didn’t* and *don’t* — words with apostrophes, which now insert correctly.

6 Testing and Bug Fixes (v1.1)

Before publishing, the application was reviewed and tested end to end: the API was called with 20 sample phrases and with malformed requests, and the web page was driven in a headless Chromium browser at desktop and phone widths. The issues below were found and fixed.

#	Issue	Severity	Fix
1	Clicking a suggestion containing an apostrophe (“don’t”, “it’s” — 315 such words in the vocabulary) threw a JavaScript error and inserted nothing.	High	Cards are built with DOM APIs and `textContent` + click listeners instead of an HTML string with inline `onclick`. This also removes an HTML-injection risk.

2	Often only 2 suggestions under “Top 3”: the app took the top 3 and *then* removed `<00V>` (7 of 20 test phrases were affected).	High	Walk the full ranking and skip invalid tokens until 3 real words are collected; also skip punctuation-only tokens such as a lone `'`.
3	The `"` key on the symbol keyboards threw a syntax error and showed `\"` as its label (backslash escaping does not work in HTML attributes). The backslash key showed `\\`.	High	Use the `"` entity; corrected labels; escaped `< > &`.
4	`requirements.txt` allowed TensorFlow 2.14/2.15, which cannot load this model (saved with Keras 3.11) or unpickle the tokenizer (`keras.src.legacy`). README listed different versions and Python 3.8.	High	Require `tensorflow>=2.16`, `keras>=3.11`, `Flask>=3.0`; README updated to Python 3.10–3.13.
5	Model files were opened relative to the current folder, so `python Word_Predictor/app.py` from elsewhere crashed.	Medium	Paths resolved from the location of `app.py`.
6	A JSON `null` body or a non-string `text` crashed the endpoint with HTTP 500.	Medium	Validated input; returns HTTP 400 with a message. Input is capped at 2,000 characters.
7	Pressing the physical Shift key toggled the on-screen Shift, so typing “Hello” and then tapping *w* produced “W”.	Medium	Removed the global key handlers; the on-screen Shift is only controlled by the on-screen key.
8	Backspace deleted text even when focus was elsewhere on the page, and Ctrl+Backspace deleted one letter instead of one word.	Medium	The textarea now handles physical keys natively.
9	Every on-screen key press sent an un-debounced request; slow responses could arrive out of order and show suggestions for older text.	Medium	All input is debounced; a request counter discards stale responses; Clear cancels pending requests.
10	On phones the keyboard wrapped badly (“0”, “P”, Backspace and Enter on rows of their own).	Low	Rows no longer wrap; keys shrink to fit.
11	Keys were always drawn in capitals, so Shift/Caps Lock state was not visible; Caps Lock could not be turned off with a single tap; no way back to ABC from the special-symbol keyboard.	Low	Keys show the case they will type; single tap exits Caps Lock; ABC key added.
12	`debug=True` was hard-coded (exposes the Werkzeug debugger); README's sample confidences added up to more than 100%.	Low	Debug via `FLASK_DEBUG=1`; `PORT` configurable; README uses real model output.

7 Installation and Usage

```
git clone <your-repo-url>
cd Word_Predictor
python -m venv venv
venv\Scripts\activate          # Windows   (macOS/Linux: source venv/bin/activate)
pip install -r requirements.txt
python app.py                  # open http://localhost:5000
```

Project structure

```
Word_Predictor/
├── app.py
├── requirements.txt
├── word_predictor1.ipynb
├── next_word_model.keras
├── tokenizer.pkl
├── model_config.pkl
├── templates/index.html
└── docs/Project_Report.pdf
```

Requirements

- Python 3.10 – 3.13
- TensorFlow 2.16+ with Keras 3.11+
- Flask 3.x, NumPy 1.26+

Environment variables

- `PORT` — default 5000
- `FLASK_DEBUG=1` — auto-reload during development

8 Limitations and Future Work

- **Output layer larger than needed.** The Dense and Embedding layers are sized for all 19,689 words seen, but the tokenizer only ever produces the 10,000 most frequent. About half of those weights are never used (tested: words beyond the top 10,000 receive under 0.04% of the probability). Setting `vocab_size = min(num_words, len(word_index) + 1)` would roughly halve the model.
- **No validation set.** Early stopping watches training loss, so overfitting cannot be detected. Adding `validation_split=0.1` and monitoring `val_loss` would give an honest accuracy figure.
- **Model file size.** `next_word_model.keras` is 62 MB because it also stores the Adam optimizer state (about two-thirds of the file). GitHub accepts it (limit 100 MB) but warns above 50 MB; saving an inference-only copy would bring it to about 21 MB.
- **Next word, not word completion.** A half-typed word is treated as a complete word.
- **Vintage vocabulary.** Trained on 19th-century literature, so modern words become `<OOV>`.
- **Future ideas:** temperature sampling (“creative mode”) in the UI, word completion, a larger or Transformer-based model, and multi-language support.